WaveAlert

Frontend Tesztelési Dokumentáció

Balatonmáriafürdői Vízisport Egyesület Tagnyilvántartó Rendszer

Verzió: 1.0.0 | 2026. április | Jest 29.0.0 | GitHub: github.com/ManyiKornel755/BMFVSE

1. Összefoglaló

Teszt Összefoglaló

Metrika	Érték
Összes teszt	15
Sikeres	15
Sikertelen	0
Test Suites	3/3 passed
Lefedettség cél	70%

Tesztelési Kategóriák

Kategória	Tesztek	Leírás
DOM Tesztek	5 teszt	Form elemek és validáció
API Tesztek	5 teszt	HTTP kérések és hibakezelés
UI Tesztek	5 teszt	CSS, modal-ok, DOM manipuláció

2. Projekt Áttekintés

WaveAlert Rendszer

A WaveAlert egy átfogó webes alkalmazás a Balatonmáriafürdői Vízisport Egyesület számára, amely a következő funkciókat biztosítja:

- Tagok nyilvántartása
- Eseménykezelés
- Dokumentum létrehozás

Technológiai Stack

Backend komponensek:

- Node.js
- SQL Server adatbázis
- RESTful API

Frontend komponensek:

- React (ES6+)
- HTML5
- CSS3
- Responsive design

Tesztelési eszközök:

- Jest 29.0.0
- JSDOM 22.0.0
- Axios (mocked)
- Testing Library

3. Tesztelési Stratégia

Tesztelési Megközelítés

A projekt tesztelése három fő kategóriára oszlik:

Unit Tesztek (DOM)

Alapvető DOM manipuláció és elem létezés ellenőrzése. Scope: Form elemek, input validáció, események. Eszközök: JSDOM, Jest.

Integration Tesztek (API)

Backend API kommunikáció tesztelése. Scope: HTTP kérések, hibakezelés, authentikáció. Eszközök: Axios (mocked), Jest.

Component Tesztek (UI)

Felhasználói felület működésének tesztelése. Scope: CSS osztályok, modal-ok, responsive design. Eszközök: JSDOM, Jest, DOM Testing Library.

Lefedettség Célok

Kategória	Cél
Branches	70%
Functions	70%

Lines	70%
Statements	70%

4. DOM Tesztek

Teszt 1: Root elem betöltődik

Kategória: Alapvető DOM
Prioritás: Kritikus
Futási idő: 14ms

Ez a teszt ellenőrzi, hogy az alkalmazás gyökér (root) eleme megfelelően betöltődik-e a DOM-ban, és a várt típusú (DIV) elem-e. A root elem az egész JavaScript alkalmazás alapja. Ha ez az elem nem létezik vagy hibás típusú, az alkalmazás nem fog megfelelően renderelődni.

Implementáció

```
test('Root elem betoltodik', () => {
  const rootElement = document.getElementById('root');
  expect(rootElement).toBeTruthy();
  expect(rootElement.tagName).toBe('DIV');
});
```

PASS - Root elem létezik és DIV típusú

Teszt 2: Username mező validáció kitöltött értékkel

Kategória: Input Validáció
Prioritás: Kritikus
Futási idő: 18ms

A teszt a username input mező required attribútumának helyes működését ellenőrzi kitöltött érték esetén. A felhasználónév alapú bejelentkezés ellenőrzése biztosítja, hogy az alkalmazás jelenlegi állapotának megfelelően működjön a validáció.

Implementáció

```
test('Username mezo validacio kitoltott ertekkel', () => {
  const usernameInput = document.getElementById('username');
  usernameInput.value = 'testuser123';
  const isValid = usernameInput.checkValidity();
  expect(isValid).toBe(true);
});
```

PASS - Username validáció megfelelően működik kitöltött érték esetén

Teszt 3: Form submit esemény kezelés

Kategória: Event Handling
Prioritás: Kritikus
Futási idő: 6ms

Ez a teszt ellenőrzi a form submit esemény megfelelő kezelését, beleértve a preventDefault() hívását és az event listener callback függvény végrehajtását.

Implementáció

```
test('Form submit esemeny kezeles', () => {
  const loginForm = document.getElementById('loginForm');
  let submitCalled = false;
  loginForm.addEventListener('submit', (e) => {
    e.preventDefault();
    submitCalled = true;
  });
  const submitEvent = new window.Event('submit', {
    bubbles: true, cancelable: true
  });
  loginForm.dispatchEvent(submitEvent);
  expect(submitCalled).toBe(true);
});
```

PASS - Submit esemény megfelelően kezelve, oldal nem töltődik újra

Teszt 4: Password mező validációja kitöltött értékkel

Kategória: Input Validáció
Prioritás: Magas
Futási idő: 6ms

A teszt a password input mező required attribútumának helyes működését ellenőrzi nem üres érték esetén.

Implementáció

```
test('Password mezo validacioja kitoltott ertekkel', () => {
  const passwordInput = document.getElementById('password');
  passwordInput.value = 'password123';
  const isValid = passwordInput.checkValidity();
  expect(isValid).toBe(true);
});
```

PASS - Password validáció helyes kitöltött érték esetén

Teszt 5: Input mező érték beállítása és lekérése

Kategória: DOM Manipuláció
Prioritás: Közepes

Futási idő: 5ms

Ez a teszt az input mező value property alapvető működését ellenőrzi: egy érték programozott beállítását és azt követő lekérését.

Implementáció

```
test('Input mezo ertek beallitasa es lekerese', () => {
  const usernameInput = document.getElementById('username');
  const testUsername = 'johndoe';
  usernameInput.value = testUsername;
  expect(usernameInput.value).toBe(testUsername);
});
```

PASS - Value property megfelelően működik, beállított érték visszaolvasható

5. API Tesztek

Mocking Stratégia

Az API tesztek során minden esetben Axios mocking technikát alkalmazunk. Előnyök:

- Gyorsaság: Nincs valódi HTTP kérés, a tesztek milliszekundumok alatt lefutnak.
- Megbízhatóság: Determinisztikus eredmények, nincs hálózati hibákból eredő flakiness.
- Backend függetlenség: Frontend tesztek futtathatók backend nélkül is.
- Edge case tesztelés: Könnyen szimulálhatók ritkán előforduló hibák.

Teszt 1: GET kérés sikeres

Kategória: HTTP Methods
Prioritás: Kritikus
Futási idő: 3ms

Egy alapvető GET kérés sikeres végrehajtását és a válasz megfelelő feldolgozását ellenőrzi.

Implementáció

```
test('GET keres sikeres', async () => {
  const mockData = { id: 1, name: 'Test User' };
  axios.get.mockResolvedValue({ data: mockData, status: 200 });
  const response = await axios.get('/api/users/1');
  expect(response.data).toEqual(mockData);
  expect(response.status).toBe(200);
  expect(axios.get).toHaveBeenCalledWith('/api/users/1');
});
```

PASS - GET kérés sikeres, válasz megfelelően feldolgozva

Teszt 2: POST kérés sikeres

Kategória: HTTP Methods
Prioritás: Kritikus
Futási idő: <1ms

A teszt egy POST kérés sikeres végrehajtását vizsgálja, amely login vagy regisztrációs műveleteket szimulál.

Implementáció

```
test('POST keres sikeres', async () => {
  const postData = {
    username: 'testuser', password: 'password123'
  };
  const mockResponse = {
    token: 'abc123', user: { id: 1, username: 'testuser' }
  };
  axios.post.mockResolvedValue({ data: mockResponse, status: 201 });
  const response = await axios.post('/api/auth/login', postData);
  expect(response.data).toEqual(mockResponse);
  expect(response.status).toBe(201);
});
```

PASS - POST kérés sikeres, token és user adatok helyesen visszaadva

Teszt 3: API hiba 404 kezelés

Kategória: Error Handling
Prioritás: Kritikus
Futási idő: 1ms

A 404 Not Found HTTP hiba megfelelő kezelését vizsgálja, amely akkor fordul elő, amikor egy nem létező erőforrást próbálunk lekérni.

Implementáció

```
test('API hiba 404 kezeles', async () => {
  const errorMessage = 'Not Found';
  axios.get.mockRejectedValue({
    response: { status: 404, data: { message: errorMessage } }
  });
  try {
    await axios.get('/api/users/999');
  } catch (error) {
    expect(error.response.status).toBe(404);
    expect(error.response.data.message).toBe(errorMessage);
  }
});
```

PASS - 404 hiba megfelelően kezelve, státusz és üzenet ellenőrizve

Teszt 4: JSON response feldolgozás

Kategória: Data Processing
Prioritás: Magas
Futási idő: <1ms

Összetett, nested JSON objektumok helyes feldolgozását és parsing-át ellenőrzi.

Implementáció

```
test('JSON response feldolgozas', async () => {
  const jsonData = {
    users: [{ id: 1, name: 'User 1' }, { id: 2, name: 'User 2' }]
  };
  axios.get.mockResolvedValue({ data: jsonData, status: 200 });
  const response = await axios.get('/api/users');
  const parsedData = response.data;
  expect(parsedData.users).toHaveLength(2);
  expect(parsedData.users[0].name).toBe('User 1');
});
```

PASS - JSON objektum helyesen feldolgozva, nested adatok elérhetők

Teszt 5: Authorization header hozzáadása token-nel

Kategória: Authentication
Prioritás: Kritikus
Futási idő: 1ms

A Bearer token alapú autentikáció helyes implementációját ellenőrzi: a JWT token megfelelően kerül-e hozzáadásra a HTTP header-ekhez.

Implementáció

```
test('Authorization header hozzaadasa token-nel', async () => {
  const token = 'bearer-token-123';
  const mockConfig = {
    headers: { Authorization: `Bearer ${token}` }
  };
  axios.get.mockResolvedValue({ data: {}, status: 200 });
  await axios.get('/api/protected', mockConfig);
  expect(axios.get).toHaveBeenCalledWith('/api/protected', mockConfig);
});
```

PASS - Authorization header helyesen beállítva Bearer token-nel

6. UI Tesztek

Teszt 1: Navigáció menü megjelenik

Kategória: Navigation
Prioritás: Magas

A navigációs menü (navbar) megfelelő betöltődését és struktúráját ellenőrzi.

Implementáció

```
test('Navigacio menu megjelenik', () => {
  const navbar = document.querySelector('.navbar');
  expect(navbar).toBeTruthy();
  expect(navbar.children.length).toBe(2);
});
```

PASS - Navbar elem létezik 2 gyerek elemmel

Teszt 2: CSS osztály alkalmazása gombra

Kategória: CSS Classes
Prioritás: Közepes

A teszt ellenőrzi, hogy a .btn-primary CSS osztály megfelelően alkalmazva van-e egy gomb elemen.

Implementáció

```
test('CSS osztaly alkalmazasa gombra', () => {
  const button = document.getElementById('openModal');
  expect(button.classList.contains('btn-primary')).toBe(true);
});
```

PASS - btn-primary osztály alkalmazva a gombon

Teszt 3: Modal megnyitása active osztály hozzáadásával

Kategória: Modal Dialog
Prioritás: Magas

A modal megjelenítési mechanizmusát teszteli, amely CSS osztály hozzáadásán alapul.

Implementáció

```
test('Modal megnyitasa active osztaly hozzaadasaval', () => {
  const modal = document.getElementById('testModal');
  modal.classList.add('active');
  expect(modal.classList.contains('active')).toBe(true);
});
```

PASS - Active osztály hozzáadása modal megnyitásához működik

Teszt 4: Gomb kattintás modal megnyitásra

Kategória: Event Handling
Prioritás: Magas

A gomb kattintás és modal megnyitás közötti interakciót vizsgálja, egy teljes user flow-t szimulál.

Implementáció

```
test('Gomb kattintas modal megnyitasra', () => {
  const button = document.getElementById('openModal');
  const modal = document.getElementById('testModal');
  button.addEventListener('click', () => {
    modal.classList.add('active');
  });
  button.click();
  expect(modal.classList.contains('active')).toBe(true);
});
```

PASS - Gomb kattintás megnyitja a modalt

Teszt 5: Dinamikus tartalom hozzáadása

Kategória: DOM Manipuláció
Prioritás: Közepes

Új DOM elemek programozott létrehozását és a DOM fához való hozzáadását vizsgálja.

Implementáció

```
test('Dinamikus tartalom hozzaadasa', () => {
  const container = document.querySelector('.container');
  const newElement = document.createElement('p');
  newElement.textContent = 'Uj tartalom';
  newElement.id = 'dynamicContent';
  container.appendChild(newElement);
  const addedElement = document.getElementById('dynamicContent');
  expect(addedElement).toBeTruthy();
  expect(addedElement.textContent).toBe('Uj tartalom');
});
```

PASS - Dinamikus tartalom sikeresen hozzáadva a DOM-hoz

7. Telepítési Útmutató

Rendszerkövetelmények

- Node.js v16.0.0 vagy újabb verzió
- npm v7.0.0 vagy újabb verzió

- Git verziókezelő rendszer

1. lépés: Repository klónozása

```
git clone https://github.com/ManyiKornel755/BMFVSE.git
cd BMFVSE/Frontend_Test
```

2. lépés: Függőségek telepítése

```
npm install
```

A telepített package-ek:

- jest v29.0.0 - Test framework
- jest-environment-jsdom v29.0.0 - DOM szimuláció
- jsdom v22.0.0 - Virtual DOM implementáció
- axios v1.4.0 - HTTP client (mocking célokra)
- @testing-library/dom v9.0.0 - DOM testing utilities
- @testing-library/jest-dom v6.0.0 - Custom Jest matchers

3. lépés: Konfiguráció ellenőrzése

```
module.exports = {
  testEnvironment: 'jsdom',
  coverageThreshold: {
    global: { branches: 70, functions: 70, lines: 70, statements: 70 }
  }
};
```

4. lépés: Tesztek futtatása

```
npm test
```

Várt kimenet:

```
PASS  ./api.test.js
PASS  ./ui.test.js
PASS  ./dom.test.js
Test Suites: 3 passed, 3 total
Tests:       15 passed, 15 total
```

8. Futtatási Parancsok

Parancs	Leírás
npm test	Összes teszt egyszeri lefuttatása
npm run test:watch	Automatikus újrafuttatás fájl módosításkor
npm run test:coverage	Kód lefedettség mérése és riport készítése

npm run test:verbose	Részletes kimenet minden tesztről
npm test dom.test.js	Csak a DOM tesztek futtatása
npm test api.test.js	Csak az API tesztek futtatása
npm test ui.test.js	Csak az UI tesztek futtatása

9. CI/CD Integráció

GitHub Actions Workflow

A következő GitHub Actions workflow automatikusan futtatja a teszteket minden push és pull request esetén. Hozzon létre egy új fájlt a .github/workflows/frontend-tests.yml útvonalon.

Pre-commit Hook Husky-val

Automatikus teszt futtatás minden commit előtt:

```
npm install --save-dev husky
npx husky install
npx husky add .husky/pre-commit "cd Frontend_Test && npm test"
```

10. Hibaelhárítás

Hiba	Ok	Megoldás
Cannot find module 'jsdom'	A jsdom package nincs telepítve	npm install --save-dev jsdom@22.0.0
document is not defined	JSDOM test environment nincs beállítva	testEnvironment: 'jsdom' a jest.config.js-ben
Timeout exceeded	Aszinkron teszt túl sokáig fut	testTimeout: 10000 a jest.config.js-ben
Coverage threshold not met	Lefedettség nem éri el a 70%-ot	Nyissa meg a coverage HTML riportot és írjon teszteket a lefedetlen részekre

11. Következő Lépések

Rövid távú fejlesztések (1-2 hét)

- E2E tesztek írása Cypress-szel
- Visual regression tesztek Percy vagy Chromatic eszközökkel
- Performance tesztek Lighthouse integrációval

- Accessibility audit axe-core library-val (WCAG 2.1 AA)

Közép távú fejlesztések (1-2 hónap)

- Backend integration tesztek valós adatbázis kapcsolattal
- Load testing Apache JMeter vagy k6 eszközökkel
- Security testing OWASP ZAP eszközzel
- Cross-browser testing BrowserStack platformon

Hosszú távú fejlesztések (3-6 hónap)

- Automatizált regressziós teszt suite kiépítése
- Continuous deployment pipeline staging környezetbe
- Sentry vagy DataDog integráció production monitorozásra
- A/B tesztelési keretrendszer implementálása